

12th Progress Report

W. Chan

Date:	28/05/2024
Subject:	RL for Flying-V
Supervisors:	Dr. E.J. van Kampen
Project term:	01/2024 - 08/2024
E-mail:	w.y.chan@student.tudelft.nl

Outcome of the previous meeting

- Reviewed mid-term/preliminary thesis. Submitted mid-term form to spa-lr. Discussed various results of the thesis.
- Talked about doing statistical testing on the obtained statistics.
- Talked about trying to install/get a nonlinear model to work before deciding on using a linear model.
- Discussed the policy colour plots.
- Discussed the utility of making a robust controller for comparison, that making this comparison would not be useful since in the best case we'd only find the previously expected result, that robust controller don't adapt as well but RL controller adapts... Brings the question of the relevancy of the last research question.
- Talked about citing sources for the control and stability coefficients for the short period model.

Description of the results since last meeting

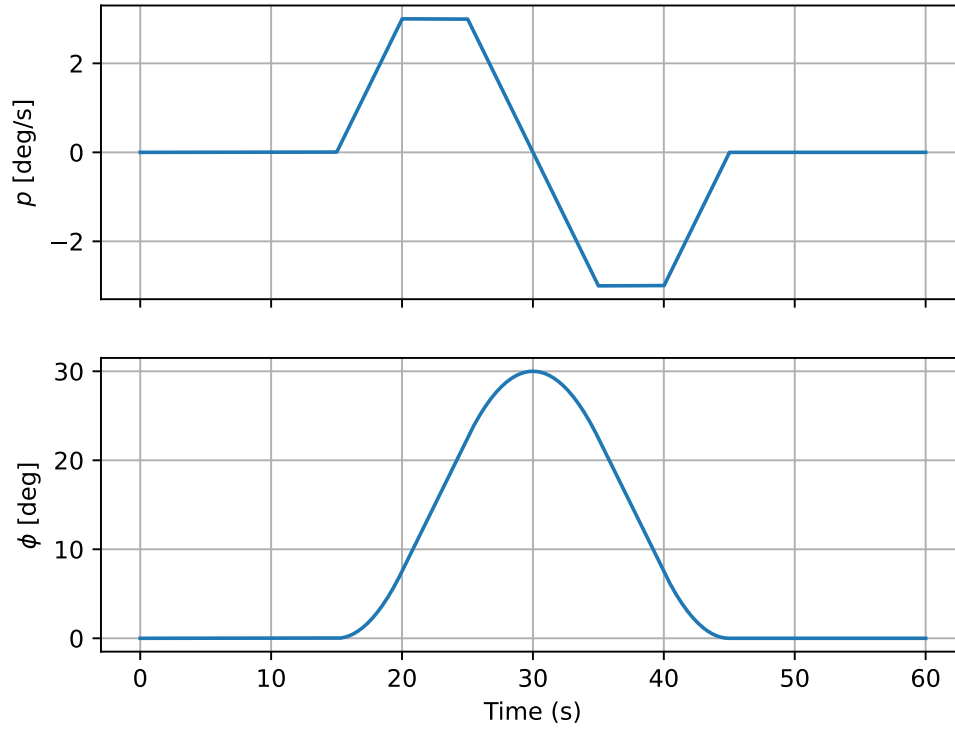
- Updated various figures to improve interpretability:
 - Used the same vertical axis range for all the actor gradient plots, likewise for critic gradient plots.
 - Changed limits of the metric axis in boxplots to start from zero, instead of zoomed in around the sampled metrics.
- Added statistical testing results (student's t -test and Vargha & Delaney's a -test). t -test results shows that the difference between statistics of baseline IDHP and all augmented/derivative algorithms are statistically significant, with the following exceptions:
 1. Difference in settling time between IDHP and IDHP(λ) is not statistically significant ($p = 0.64$), the settling times of these two can be said to be equal.
 2. For the shift cg fault. Difference in transient error between IDHP and MIDHP ($p = 0.08$), the transient error of MIDHP is worse than IDHP.

3. For the reversed elevator fault. Difference in transient error between baseline and derivative algorithms are not statistically significant for any pairs. The distribution of errors is thus similar but with the derivative algorithms having worse performance. Note here that a better comparison between the algorithms in this fault case is in number of stable controllers, as it was observed that large portions of runs for all four algorithms resulted in unstable controllers (ranging from 26% unstable in IDHP to 98% unstable for IDHP(λ)).
- Updated texts to improve the clarity of results.
 - Added explanation of t -test and a -test.
 - Added discussion of statistical significance of observed differences between sample groups.
 - Elaborated on the explanation for the colour plots showing the control policy over time.
 - Added ranking of controllers for each experiment
 - Added in definitive conclusion at the end stating which controllers are best using the rankings of algorithms.
 - Talked to Coen about citing some sources for the coefficients used in the thesis. He said that he does not remember any publications from him or the group where these coefficients were used. However, he gave an alternative suggestion for how to introduce these coefficients: I should state that these coefficients were obtained from a nonlinear flight dynamics model of the Cessna Citation 2 by linearizing this model around a chosen operating point.
 - Scheduled a meeting with Spilios on 29th May to get his take on my last research question (comparing the performance of IDHP controllers to PID controllers).
 - Had a good go at compiling the Simulink model and running it in Python, long story short: it was a success. Long story long:
 1. Realised the existing trim conditions for the model were not very trim, all states would drift away from the starting value readily.
 2. Tried making a script for auto-trimming, but for some reason, Simulink refuses to use the initial state values that I define in the workspace, so could not automate it.
 3. Manually trimmed the Citation as well as I could, the states would initially oscillate for 5 to 10 s, but then plateau (max pitch rate reached is 0.58 deg/s).
 4. Went through the updated CitAST to Python document and compiled the Simulink model.
 5. Compilation from Simulink to C code went without a hitch. But getting the python executable .pyd file threw an error, “no table3.c file found”...
 6. Saw that the Simulink model uses this table S-function for finding some coefficients, had no idea what this table thing is, but saw that it took in some non-zero input and always spat out zeroes.
 7. Looked for table3.c in the sent files, but could not find it.
 8. Decided to code a dummy S-Function myself, which takes in some variables and spits out a zero (lol).
 9. This fix worked, can get .pyd files now.
 10. Comparison of .pyd and Simulink responses are identical, compilation verified, see
 11. Messaged Olaf asking about table3, he tells me it is simply in the default MATLAB directory. Didn't use that c file at the end, because verification already passed.

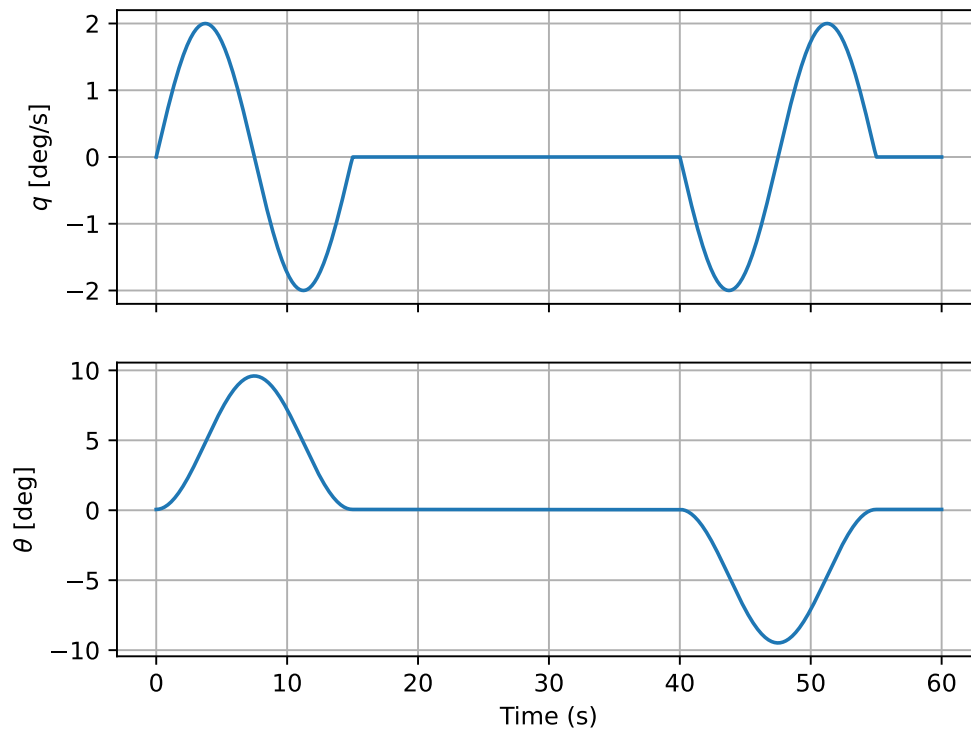
- **Test Manoeuvres/Reference signals**

For now, decided on doing rate tracking control for very simple manoeuvres. Pitch up at the start of the flight, roll to one side mid-flight, pitch down at the end of the flight. Yaw rate reference signal is kept at zero throughout the flight. The pitch and rolling manoeuvres only slightly overlap towards the end, but if IDHP shows success in this case then I can try doing more overlap. Further, could maybe try an attitude-tracking task later, if rate control seems fine, but given time constraints it is unlikely that I will have time to do both. See Figure 1 for the designed signals.

In short, I will first try rate tracking to produce a minimal working order of the algorithms and to have a safety net of results; then I will try out attitude tracking and make these my primary results *if I have time to flesh out these results*.



(a) Roll reference signal.



(b) Pitch reference signal.

Figure 1: Designed manoeuvres/reference signals.

- MDP state+action space design

Decided to first test using tracking errors as the MDP states, just like in research phase 1.

The action space is the 3 control surface deflections: aileron, elevator, and rudder.

BUT, i will try a longitudinal lateral controller split that Stefan Heyer did, I would expect that this works better than a fully coupled controller.

- **Environment Implemented**

Having obtained the nonlinear executable, designed the manoeuvres, and decided on the MDP state space, a working implementation of the MDP environment is implemented. Made a class which has a step function, taking in the 3 surface deflections, simulates the Citation's response to it, and returns the longitudinal controller's reward value and gradient (pitch rate tracking error), plus the lateral controls reward value and gradient (roll and yaw rate tracking error). The function returns their states as well, but that is actually only useful for visualization or debugging, it is not "core" to the algorithms.

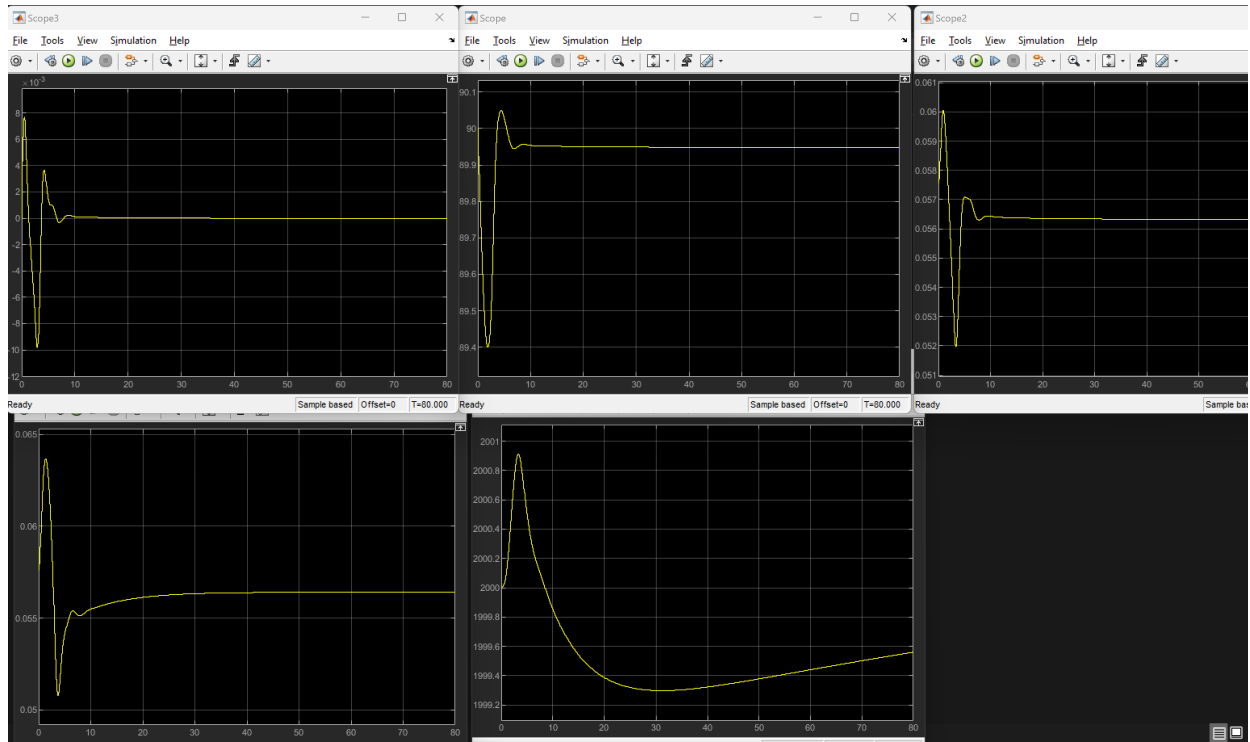
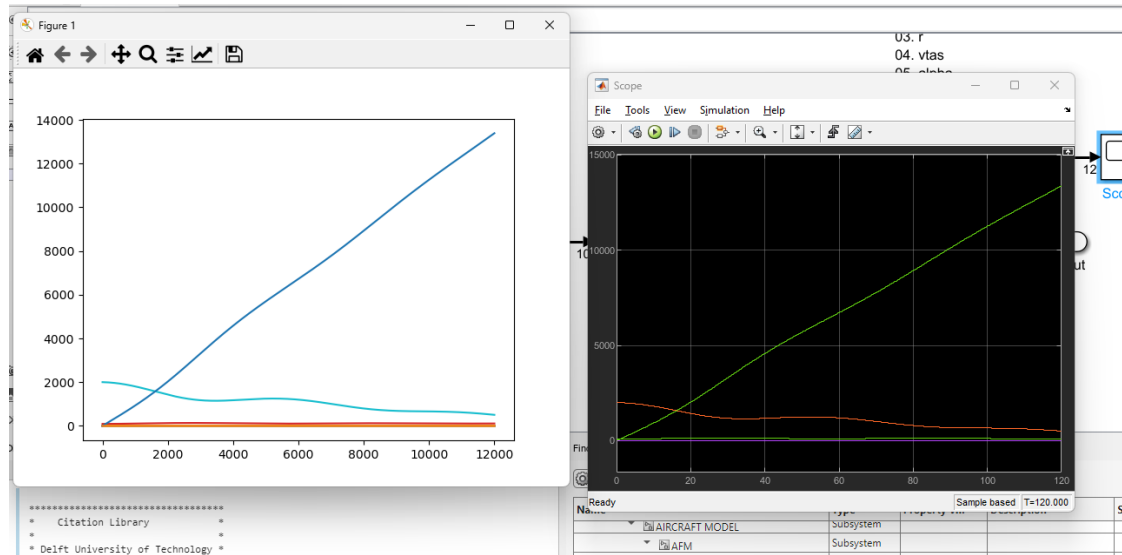
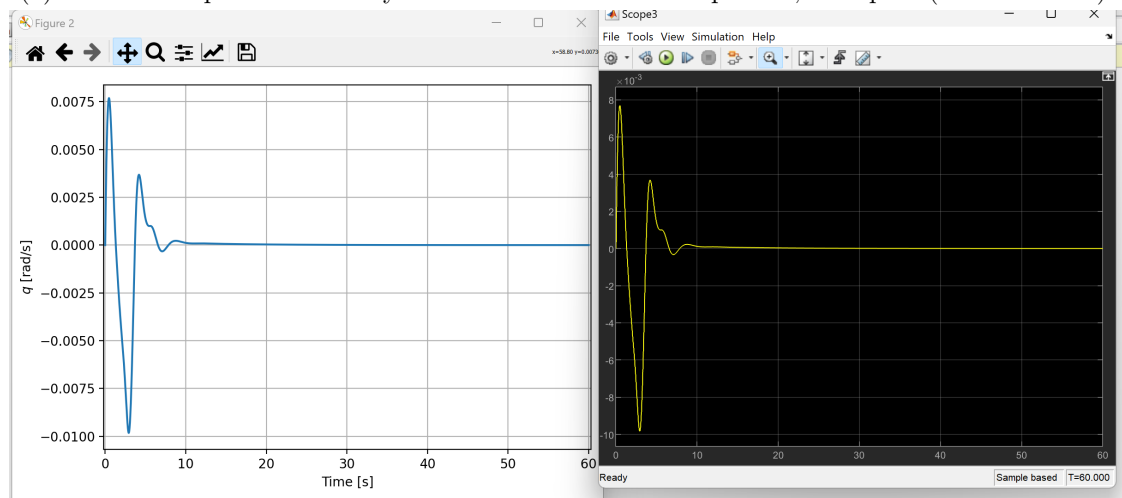


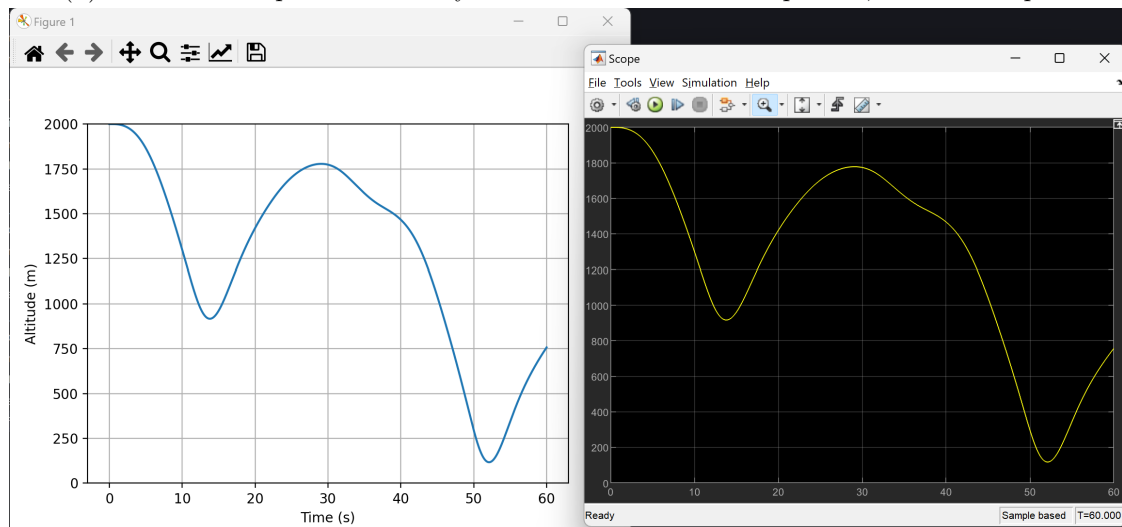
Figure 2: Trimmed model's state responses, states presented (starting top left and going clockwise) are: q [rad/s], V_{TAS} [m/s], α or θ [rad], α or θ [rad], and h [m].



(a) All state responses of the Python and Simulink models plotted, no inputs (not even trim).



(b) Pitch rates responses of the Python and Simulink models plotted, with trim input.



(c) Altitude responses of the Python and Simulink models plotted, trim plus sine elevator input.

Figure 3: Model response comparison between the Python and Simulink models to various inputs.

Plans for next 2 weeks

1. Flesh out/verify the implementation of the citation environment.

2. Rework my IDHP implementation so that the actor and networks can have an arbitrary number of hidden neurons, inputs, and outputs without hard coding. Or maybe i just resort to hard coding...
3. Use the reworked IDHP on the designed flight control problems, and do hyperparameter tuning.

Questions to discuss if we have time in meeting

1. What companies in the industry do work related to applying RL to flight control? Or what are some big research groups/labs that are working on this front.
2. Clarification on product of second research phase. Are the main expected products: Research paper, full thesis containing intro, part 1 (research paper), part 2 (literature study and preliminary results), part 3 (additional results if any), and conclusion, produced code made public basically?
3. How does a cascaded policy function work? I am confused about how to set up the update equations for the two actors. E.g., do i use the “total” actor reward for both the inner and outer actors, such as pitch rate tracking error? Using the following generic update equation for an actor:

$$\Delta w_a = \left(\frac{\partial r}{\partial x} + \gamma \lambda(s) \frac{\partial x}{\partial a} \right) \frac{\partial a}{\partial w_a} \quad (1)$$

And then therefore only the $\frac{\partial a}{\partial w_a}$ term in the update equation is different for the two networks (same $\left(\frac{\partial r}{\partial x} + \gamma \lambda(s) \frac{\partial x}{\partial a} \right)$ term for both networks)?

4. Does sending the trim input to the aircraft achieve the exact same effect as finding a trim actor network? Am I missing something about the trim actor network?
5. Strictly speaking from theory, should the critic for IDHP be derivative of value function with respect to MDP states? I am asking this because sometimes people have the IDHP output be w.r.t. aircraft states even though they use different states for the network input (the MDP state). And also for DHP literature everyone just uses the same state variables for MDP and model.
6. Any suggestions on interesting topics to explore in this second research phase?

A New explanation of the policy colour plots in thesis

(Regarding the reversed elevator fault and difficulty for algorithms to control it) To investigate this matter, it would be useful to understand what the policy function looks like for a controller which is stable versus one which is unstable. Such a study is simple to conduct if only one instantaneous policy is studied, or if the policy function does not adapt and change over time. However, the policy function in the IDHP algorithm does evolve over time, with the policy function producing a different mapping at every time step of the simulation. Therefore, to be able to represent this temporal variation clearly, a scalar field plotted as a colour map is used. Specifically, at every time step, the policy function over the domain $e = [-5, 5]$ deg is recorded, by determining what the elevator deflection is over this domain of errors. This record is done at every single timestep, and the result can be plotted as a scalar field where the x and y axes are t and e , and the z axis is δ_e . The z axis is then mapped to a range of colours to represent the magnitude of elevator deflection.